CHENNAI INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Autonomous)

PROJECT-BASED LEARNING (PBL)
Mobile Application Development (CS4504) | Academic Year 2026–27

GROUPDEAL

BUY TOGETHER, SAVE MORE

LOCALITY-BASED GROUP BUYING WITH DYNAMIC PRICING & SUPPLIER BIDDING

PROJECT TEAM
ARUL MURUGAN M – 24CS0084
RITVIK DODDAPANENI – 24CS0755

FACULTY GUIDE
Mrs. M. Sindhu
Assistant Professor, Dept. of CSE

1. ABSTRACT

GroupDeal is an Android application for locality-based group buying. Residents of the same area join a shared buying group; each new member raises the group quantity, and a tier-based pricing engine moves every member to the lower unit price reached. Suppliers then compete through reverse-auction bids, and members pay securely and track their orders. Built with Kotlin, Jetpack Compose and MVVM on a real-time Firebase backend with Razorpay payments and push alerts.

2. PROBLEM STATEMENT

- Individual buyers pay retail prices; bulk discounts reach only high-volume buyers.
- Neighbours with the same needs order separately and cannot pool demand.
- Buyers cannot see how many more participants would unlock a lower price.
- There is no simple way to signal local demand for products not yet sold.

3. OBJECTIVES

01	Enable locality-based group buying (GPS / pincode) ✓
02	Lower the unit price automatically as members join ✓
03	Let suppliers compete through reverse-auction bids ✓
04	Turn upvoted demand requests into group deals ✓
05	Provide secure payments and live order tracking ✓
06	Notify members in real time and build trust ratings ✓

4. TECHNOLOGIES USED

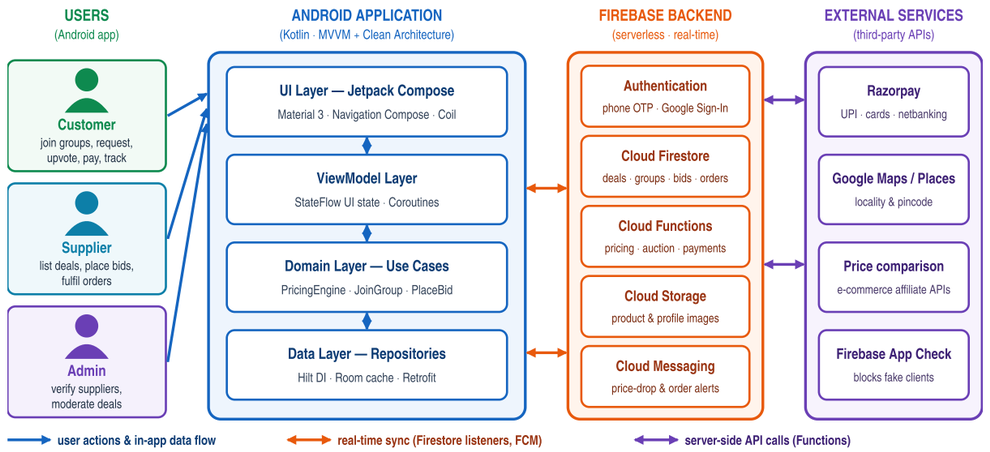
Kotlin — language	Jetpack Compose — declarative UI	Material 3 — design system	MVVM + Hilt — architecture, DI	Coroutines, Flow — async, StateFlow
Room — offline cache	Retrofit, OkHttp — REST client	Coil — image loading	Firebase Auth — OTP, Google	Cloud Firestore — real-time DB
Cloud Functions — server logic	Cloud Messaging — push alerts	Razorpay — payments	Google Maps — locality	JUnit, MockK — testing

5. METHODOLOGY

01 Requirement Analysis → 02 UI / UX Design (Material 3) → 03 Android App (Kotlin + Compose) → 04 Backend & Cloud (Firebase) → 05 Testing & Integration → 06 Deployment (Play Console)

6. SYSTEM ARCHITECTURE

MVVM + Clean Architecture · Hilt DI · real-time Firestore sync



Razorpay orders are created and payment signatures verified in Cloud Functions — never on the device.

7. DATA & INPUTS

- **Firestore collections:** users, localities, deals, groups, requests, bids, orders, payments.
- **Deals:** base price, MRP and member-count price tiers set by the supplier.
- **Locality:** GPS location or pincode, mapped via Google Maps / Places.
- **User inputs:** join / leave, upvote, request form, bids, payment, ratings.



Data flow:

→ User signs in (OTP) and sets locality.
→ Joins a group in a Firestore transaction.
→ Cloud Function recomputes tier and price.
→ FCM alerts every member of the drop.
→ Suppliers bid; members pay via Razorpay.

8. IMPLEMENTATION

server-side in Cloud Functions

Input: base price, tiers (min. members → price), members n → Sort tiers: by min. members, highest first → Select tier: first tier with n ≥ threshold → Unit price: tier price, else base price → Display: price, saving % vs MRP → Next tier: members needed = m_{next} − n

Worked example — "A2 Gir Cow Ghee 1 L" (MRP ₹1,400; quantity-discount schedule [3])

₹1,250	₹1,099	₹949 · now	₹799
base · 1–4	tier 1 · 5+	tier 2 · 15+ · 18 members, 32% below MRP	tier 3 · 30+ · 12 more

- Runs inside a Firestore transaction on every join / leave, so simultaneous joins never corrupt the count [6].
- After the target is reached, a reverse auction picks the lowest supplier bid [4]; market prices are shown only for comparison.

9. APPLICATION SCREENSHOTS

insert device / emulator screenshots



Discover Deals — locality, search, categories
Group Detail — price tiers, join, checkout
Demand Requests — request & upvote
My Groups — joined groups, live price

10. RESULTS & KEY OUTCOMES

Group pricing — price steps down per tier as members join

Secure & trusted — OTP sign-in, verified payments, ratings

Real-time sync — every member sees price drops instantly

Feature	Status	Remarks
Locality groups	✓	GPS / pincode; nearby deals only
Deal discovery	✓	search, category filters, offline cache
Join / leave group	✓	atomic Firestore transaction
Tier-based pricing	✓	price drops as each tier is reached
Supplier bidding	✓	reverse auction; lowest bid wins
Demand requests	✓	upvoted requests become group deals
Payments	✓	Razorpay; verified in Cloud Functions
Order tracking	✓	status timeline with FCM alerts
Trust & ratings	✓	member and supplier reputation tiers
Price comparison	✓	market price vs group price

11. DISCUSSION / KEY FINDINGS

- A member-count step function makes the incentive visible: one join lowers the price for every member [1].
- Showing members still needed makes the next tier a shared goal [2].
- Server-side pricing in transactions keeps prices correct under concurrent joins [6].
- Reverse auctions let suppliers compete only after demand is proven [4].
- Clean MVVM layers keep the app testable and maintainable [5].

12. CONCLUSION

GroupDeal shows how locality-based group buying can bring bulk prices to individual buyers. Tier-based dynamic pricing, supplier reverse auctions, demand requests, secure payments and real-time alerts work together on a modern Android and Firebase stack, giving neighbourhoods a transparent, community-driven way to buy together and save more.

13. FUTURE ENHANCEMENTS

- Demand forecasting from request history
- Tamil and Hindi language support
- Web dashboard for suppliers and admins
- Delivery partner integration
- Referral rewards for inviting neighbours

14. REFERENCES

[1] K. S. Anand and R. Aron, "Group buying on the web: A comparison of price-discovery mechanisms," *Management Science*, vol. 49, no. 11, pp. 1546–1562, 2003.
[2] R. J. Kauffman and B. Wang, "New buyers' arrival under dynamic pricing market microstructure: The case of group-buying discounts on the Internet," *J. Management Information Systems*, vol. 18, no. 2, pp. 157–188, 2001.
[3] R. J. Dolan, "Quantity discounts: Managerial issues and research opportunities," *Marketing Science*, vol. 6, no. 1, pp. 1–22, 1987.
[4] S. D. Jap, "An exploratory study of the introduction of online reverse auctions," *Journal of Marketing*, vol. 67, no. 3, pp. 96–107, 2003.
[5] Android Developers, "Guide to app architecture," developer.android.com/topic/architecture.
[6] Firebase Documentation, "Transactions and batched writes," firebase.google.com/docs/firestore/manage-data/transactions.

"Buy Together, Save More — more neighbours in the group, a lower price for every member."